

CredTrack

AI-Powered Credit Card & Utility Bill Tracker

Course: INFO 6350 — Smartphones-Based Web Development

Professor: Dr. Rabah Ahmed

Northeastern University

College of Engineering

Semester: Spring 2026

Student: Siddharth Yogesh Nashikkar

1. Introduction

CredTrack is a full-stack iOS application that automatically tracks credit card statements, due dates, individual transactions, and utility bills — without the user ever needing to log in to a banking portal. The app connects to the user's Gmail account using Google OAuth2 with read-only scope, and a backend actor system continuously scans for new statement and bill emails in the background. When a statement email or PDF attachment arrives, a locally-hosted LLM (Llama 3.1 via Ollama) extracts every relevant field — balance, due date, minimum payment, and individual transactions — and persists them to the database. The iOS client presents all of this data in a clean, dark-mode SwiftUI interface built on the CRED Synth / NeoPop design system.

The backend is a distributed system: a Spring Boot REST API serves the iOS client, while a parallel Scala Akka Typed actor system handles all asynchronous processing. Actors are supervised, event-sourced, and cluster-sharded across two JVM nodes — making the pipeline resilient to partial failures without any data loss.

2. System Architecture

2.1 iOS Client — SwiftUI MVVM

The iOS app is organised across four main tabs: Cards, Utility, Analysis, and Profile. Each screen is a SwiftUI view backed by a dedicated `@StateObject` ViewModel. A shared `AppStateManager` (injected as `@EnvironmentObject`) holds the authenticated user session and card list so every view can react to global state changes without prop-drilling. Navigation uses `NavigationStack` with typed navigation values for type-safe routing. All network calls go through a singleton `APIClient` that attaches a Firebase ID token (fetched fresh via `Auth.auth().currentUser.getIdToken()` on each request) as the Bearer token. On a 401, the client throws `APIError.unauthorized` and the app re-triggers the login flow.

- 25+ SwiftUI views across 4 tab sections
- `async / await` networking via `APIClient` singleton — no callback pyramids
- `@StateObject` per screen for isolated, testable view state
- `NavigationStack` with typed values — back stack is always deterministic
- Firebase ID token on every request; silent refresh on 401

2.2 Backend — Spring Boot REST API

The Spring Boot layer is stateless and JWT-secured. It exposes all endpoints consumed by the iOS client: card management, statement queries, transaction listing, PDF upload, payment recording, and analytics. It also exposes `/internal/*` endpoints used exclusively by the AI agent actor system to write extracted data back to PostgreSQL. BIN lookup uses a pre-loaded `BinRecordRepository` — 8-digit BIN tried first, falling back to 6-digit if no match — and returns the card issuer name to the iOS app without any external API call.

2.3 AI Agent — Akka Typed Actor System

The Akka Typed actor system (written in Scala) runs as a separate JVM process alongside the Spring Boot API. It owns all asynchronous, long-running work: Gmail polling, LLM extraction, event sourcing, and analytics computation. Actors are supervised with `restartWithBackoff` (2 s $\rightarrow$ 60 s) and communicate exclusively through typed message protocols — no shared mutable state.

The core actors and their responsibilities:

- `EmailPipelineCoordinator` — root coordinator; triggered by `EmailPollScheduler` every hour; spawns child actors per cycle
- `EmailFetcherActor` — calls Gmail History API with the stored `historyId` cursor; returns only new messages since last poll
- `InitCardScanActor` — spawned once per card when `gmailScanComplete=false`; does keyword search for historical statements
- `StatementExtractorActor` — sends email/PDF text to Llama 3.1 via Ollama; parses structured JSON response
- `PaymentExtractorActor` — parses payment confirmation emails (Chase-specific patterns) to update paid status
- `LlmTransactionExtractorActor` — extracts individual transactions from Gmail alert emails
- `StatementWriterActor` — POSTs extracted statements to `/internal/statements` (idempotent via `gmail_message_id` unique constraint)
- `PersistentStatementLedgerActor` — event-sourced; maintains an in-memory ledger of all statements + transactions; replays journal on crash
- `AnalyticsAggregatorActor` — reads ledger state; computes per-card and per-utility analytics on demand
- `AnalyticsWorkerActor` — cluster-sharded by `userId` across two JVM nodes; holds each user's analytics snapshot
- `UtilityBillCoordinatorActor` — mirrors the statement pipeline for utility bill emails (Eversource, National Grid)

2.4 Data Store

- PostgreSQL — primary store: `users`, `user_cards`, `card_statements`, `transactions`, `utility_accounts`, `utility_bills`, `gmail_credentials`
- Akka journal — event store for `PersistentStatementLedgerActor` crash recovery
- Firebase Storage — statement PDFs stored here; path saved in `card_statements.firebase_path`
- Gmail credentials table — stores AES-256-GCM encrypted refresh token and `historyId` cursor per user

3. Design Philosophy

3.1 CRED Synth Design System

CredTrack's visual language is built on CRED Synth — the open-source design system published by CRED, India's premium credit-card management platform. CRED Synth is built on top of NeoPop, CRED's interaction design philosophy that treats every UI element as a physical object. Buttons have hard 3-D drop shadows that make them appear raised off the screen. Card surfaces use sharp edges and layered depth. Every tap produces a visible press-down animation. The system is designed for high-contrast dark interfaces where financial amounts are the hero of every screen.

- SynthButton — NeoPop-style button with hard shadow and press-down animation
- NeuCard — dark card container with layered elevation for data grouping
- CreditCardView — styled card surface with chip, network logo, and masked PAN (dots + last 4)
- SynthChipButton — compact filter chip used in the Analytics tab segmented control
- WaveBackgroundView — animated gradient background on login and onboarding screens
- CTToast — lightweight toast for success and error feedback

Framework References: <https://github.com/CRED-CLUB> | [NeoPop iOS: https://github.com/CRED-CLUB/neopop-ios](https://github.com/CRED-CLUB/neopop-ios) | [Synth iOS: https://github.com/CRED-CLUB/synth-ios](https://github.com/CRED-CLUB/synth-ios)

3.2 Color System

- #0D0D0D — primary background ("Black c500" in CRED Synth terminology)
- #1A1A1A — card surface / elevated container color
- #FFFFFF — primary text and iconography
- #6C63FF — CredTrack purple accent (CTAs, active tab indicator, chart highlights)
- #666666 — secondary labels and metadata text

3.3 Typography

- Gilroy — primary UI sans-serif for all body copy and labels
- PPCirka — display serif for large headline balance amounts
- Large bold numerals for balance and credit limit — instantly scannable at a glance
- Tight letter-spacing on currency figures for column alignment in lists

4. How It Works — Key Flows

4.1 Adding a New Card

When the user taps + Add New and enters their card number, the following sequence runs:

1. The iOS app reads the first 8 digits (BIN) as the user types and calls GET /bins/{cardNumber}.
2. BinService on the backend tries an 8-digit BIN lookup in BinRecordRepository; falls back to 6 digits if no exact match.
3. The resolved issuer name (e.g. AMERICAN EXPRESS) is returned to the iOS app.
4. The app immediately calls GET /card-products?issuer={issuer} to fetch the full list of products from that issuer.

5. The Card Identification screen renders the matched card art via CreditCardView. If multiple products share the BIN, a carousel lets the user swipe to the correct one.
6. The user enters their credit limit and taps Add to Wallet. The app fires POST /user-cards with: cardProductId, lastFour, cardHolderName, creditLimit.
7. UserCardService saves the card to user_cards with gmailScanComplete=false.
8. On the very next hourly polling cycle, EmailPipelineCoordinator sees gmailScanComplete=false and spawns InitCardScanActor.
9. InitCardScanActor performs a Gmail keyword search (e.g. "from:chase.com subject:statement") to backfill all historical statements for this card.
10. Once the historical scan completes, gmailScanComplete is set to true and the card switches to incremental polling from that point forward.

4.2 How Statements Are Automatically Gathered from Gmail

This is the core pipeline that keeps the app up to date without any user action:

11. EmailPollScheduler triggers every hour (configurable via gmail.poll-interval-ms, default 3,600,000 ms).
12. It sends a Poll message to EmailPipelineCoordinator, which spawns EmailFetcherActor for this cycle.
13. EmailFetcherActor calls the Gmail History API: users().history().list() with startHistoryId set to the cursor stored in gmail_credentials.
14. The History API returns only messages added since the last cursor — not the full inbox. This is efficient and quota-safe.
15. Messages are filtered to known bank sender domains. Non-statement emails are discarded.
16. For each matched email, StatementExtractorActor is spawned. It fetches the email body and any PDF attachment.
17. PDFBox extracts the PDF text (truncated to the first 12,000 characters to fit the LLM context window).
18. The text is sent to Ollama (Llama 3.1) via Spring AI with a structured prompt asking for JSON output: {statementDate, balance, minimumDue, dueDate, transactions[]}
19. The LLM returns a JSON object. The actor parses it and validates the card's last four digits and bank name against what is stored — a mismatch sets status to WRONG_STATEMENT.
20. StatementWriterActor POSTs the validated data to POST /internal/statements. The gmail_message_id unique constraint ensures idempotency — duplicate emails never create duplicate statements.
21. If the LLM call fails at any point, the historyId cursor is NOT advanced. The same emails will be retried on the next polling cycle — zero data loss.
22. The iOS app picks up the new statement on its next GET /statements?cardId={id} refresh.

4.3 Marking a Statement as Paid

When the user pays their bill and wants to record it in the app:

23. From the Statement History screen, the user taps the statement they just paid.
24. The detail sheet shows the closing balance, statement date, minimum due, and bank.
25. The user pulls the sheet up to reveal the Mark as Paid panel.
26. They select the payment date using a date picker and type the amount paid.
27. Tapping Confirm fires POST /statements/{id}/mark-paid.

28. The backend sets `is_paid=true`, stores `paid_amount` and `payment_date` in `card_statements`, and recalculates the card's outstanding balance.
29. The iOS dashboard gauge updates on the next foreground refresh to reflect the cleared balance.

4.4 Automatic Statement from Gmail — Already Had a Card

For cards already onboarded (`gmailScanComplete=true`), statement detection is fully incremental:

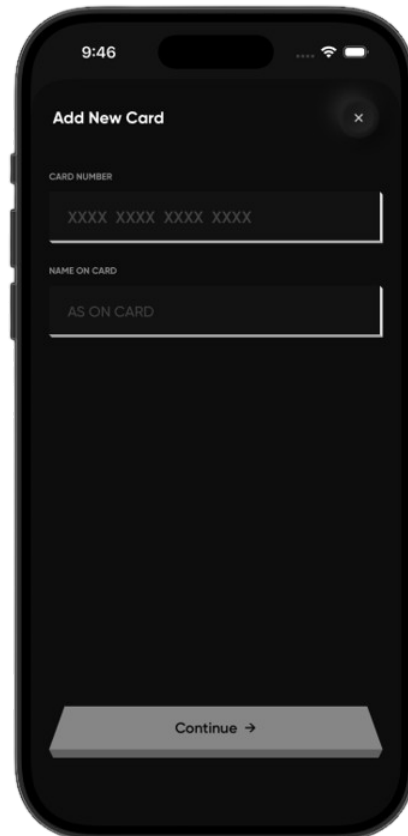
30. A new statement email arrives in Gmail (e.g. from American Express or Chase).
31. On the next hourly poll, `EmailFetcherActor` calls the History API and sees the new `messageAdded` event.
32. The email is matched against known bank domains and passed to `StatementExtractorActor`.
33. Llama 3.1 extracts the statement fields and transactions in a single LLM call.
34. `StatementWriterActor` writes the statement and all transactions to PostgreSQL via `/internal/statements`.
35. The iOS app's next background refresh automatically shows the new statement — no notification or user tap required.

5. UI Flow Walkthrough — Screen by Screen

The following screenshots were taken from the live app running on an iPhone 16 Pro. Each screen is annotated with what the user sees, what API calls are made, and what backend logic is running behind it.

Screen 1 — Add New Card

This is the first step of card onboarding. The user enters their full 16-digit card number and their name exactly as it appears on the card. As the user types, the iOS app reads the first 8 digits (the BIN — Bank Identification Number) and silently calls `GET /bins/{cardNumber}` in the background. The BIN service on the backend performs an 8-digit lookup first, falling back to 6 digits if no match is found, and returns the issuer name (e.g. AMERICAN EXPRESS). Only the last four digits of the card number are ever stored in the database — the full number is discarded after the BIN lookup completes.

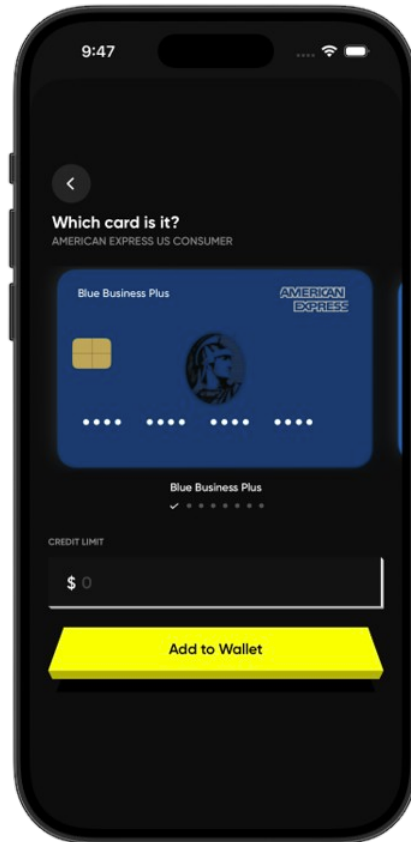


What you can do on this screen:

- Enter the full card number — BIN lookup fires automatically from the first 8 digits
- Provide the cardholder name as it appears on the physical card
- Tap Continue to advance to the card identification screen

Screen 2 — Card Identification

After the BIN is resolved, the app calls `GET /card-products?issuer={issuer}` to fetch all known products from that issuer. The matched card is displayed with its full card art — here the Amex Blue Business Plus in navy blue — using the `CreditCardView` component from the CRED Synth design system. If multiple products share the same BIN prefix, the dot-indicator carousel lets the user swipe to the correct one. The user enters their credit limit (used to calculate utilization on the dashboard) and taps Add to Wallet, which fires `POST /user-cards` with the `cardProductId`, `lastFour`, `cardHolderName`, and `creditLimit`. The backend saves the card with `gmailScanComplete=false` — this flag immediately queues a historical Gmail scan for the card on the next polling cycle.

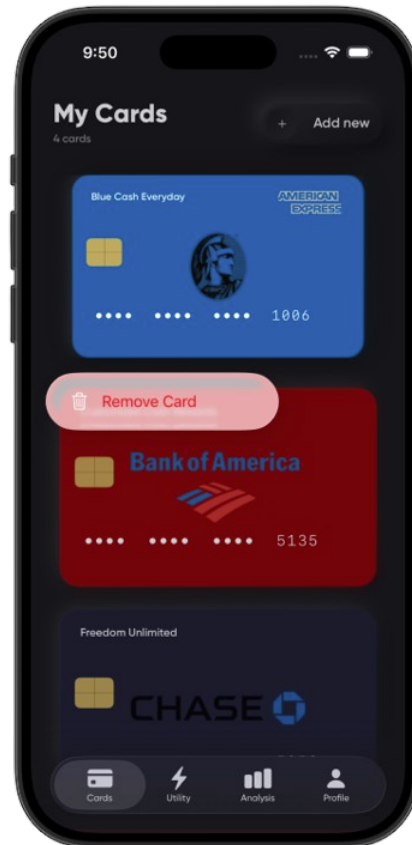


What you can do on this screen:

- View the auto-matched card art and product name resolved from the BIN
- Swipe the carousel to select the correct product if the BIN matches multiple cards
- Enter the credit limit to power the utilization gauge on the dashboard
- Tap Add to Wallet — saves the card and schedules the first Gmail statement scan

Screen 3 — My Cards Dashboard

The My Cards tab is the main dashboard, rendered on launch after the app calls `GET /user-cards` to fetch all cards for the authenticated user. Each card is displayed as a full-bleed `CreditCardView` (CRED Synth component) with the bank logo, masked PAN (dots + last four), and cardholder name. Cards are kept in sync with the backend: statement balances and current activity update on every foreground refresh. Swiping left on a card reveals a Remove Card destructive action, which calls `DELETE /user-cards/{id}` and removes the card along with all its linked statements and transactions from the database. The + Add new button in the top-right starts the BIN lookup onboarding flow.

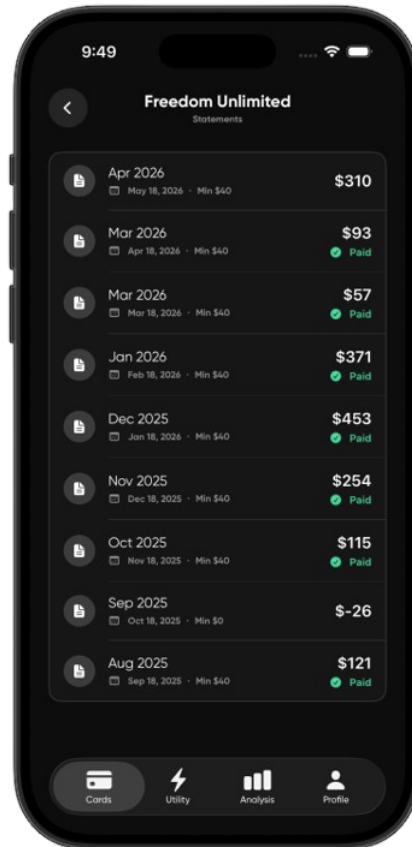


What you can do on this screen:

- See all registered cards with live balances pulled from GET /user-cards
- Tap any card to open its statement history and drill into transactions
- Swipe left to reveal Remove Card — deletes the card and all associated data
- Tap + Add new to onboard another card via BIN lookup

Screen 4 — Statement History

Tapping a card opens its statement history, fetched from GET /statements?cardId={id} with server-side pagination (page=0, size=20). Statements are sorted newest first. Each row shows the statement closing date, due date, minimum payment, and closing balance. A green Paid badge appears on statements where is_paid=true in the database. Statements are populated automatically by the Gmail extraction pipeline — when the backend's LLM reads a statement email or PDF for this card, it writes a new row to card_statements and the iOS app picks it up on the next refresh. The statement list reflects the true history the system has discovered from Gmail, not just manually entered data. A negative balance (e.g. -\$26 in Sep 2025) indicates a credit on the account.

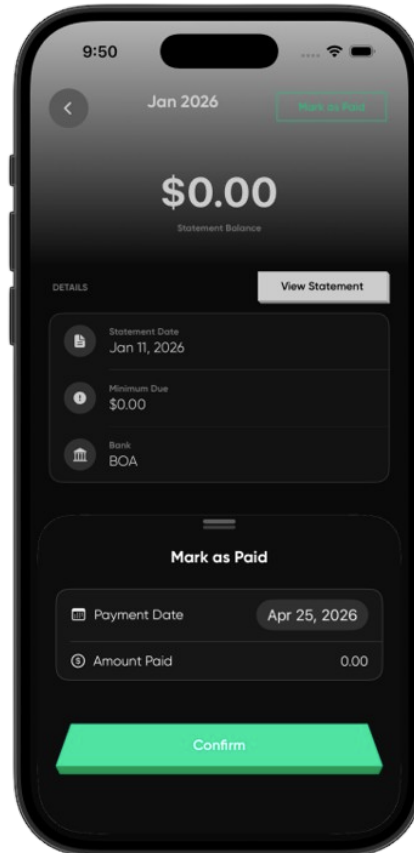


What you can do on this screen:

- Scroll through the full auto-extracted statement history for this card
- Identify paid vs unpaid statements via the green Paid badge (is_paid flag)
- See due dates and minimum payment amounts as extracted by the LLM
- Tap any statement to open its detail view and record a payment

Screen 5 — Statement Detail & Mark as Paid

Opening a statement shows its full detail: closing balance, statement date, minimum due, and the issuing bank as extracted from the email or PDF. The View Statement button calls `GET /statements/{id}/pdf` to open the original PDF from Firebase Storage if it was previously uploaded or attached to the Gmail email. Pulling the sheet up reveals the Mark as Paid panel. The user picks a payment date and enters the amount paid, then taps Confirm — this fires `POST /statements/{id}/mark-paid`, which sets `is_paid=true`, records `paid_amount` and `payment_date` in the `card_statements` table, and recalculates the card's outstanding balance so the dashboard gauge updates immediately on the next screen refresh.

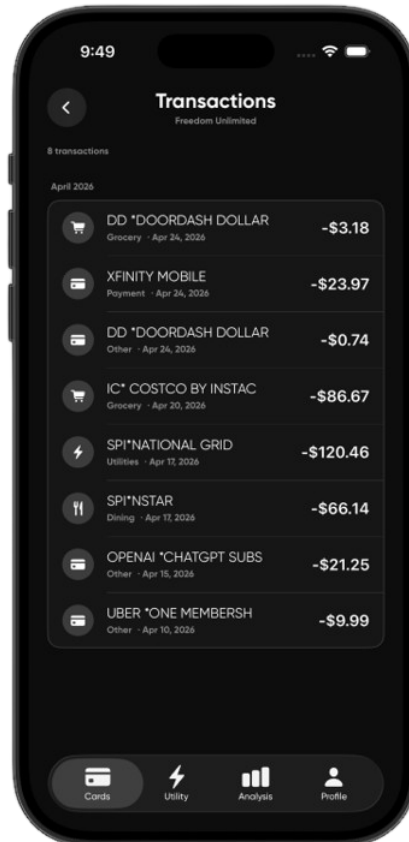


What you can do on this screen:

- View LLM-extracted details: balance, date, minimum due, bank name
- Open the original statement PDF from Firebase Storage via View Statement
- Record a payment by picking a date and amount in the Mark as Paid panel
- Confirm payment — updates `is_paid`, `paid_amount`, and refreshes the dashboard balance

Screen 6 — Transaction List

The transaction list is fetched from `GET /transactions?cardId={id}` and shows every individual charge the LLM extracted for this card, grouped by month. Transactions come from two sources: statement PDF extraction (where the PDF text is chunked to 12,000 characters and sent to Llama 3.1 via Ollama, which returns a structured JSON with a transactions array) and individual Gmail transaction alert emails processed by the `LlmTransactionExtractorActor`. Each row shows the merchant name as the LLM parsed it, the merchant category with a matching icon, the posting date, and the debit amount. When PDF extraction runs, any prior Gmail-sourced transactions for that statement period are replaced by the more complete PDF-extracted set (synthetic `gmailMessageId`: `PDF-{statementId}-{idx}`).

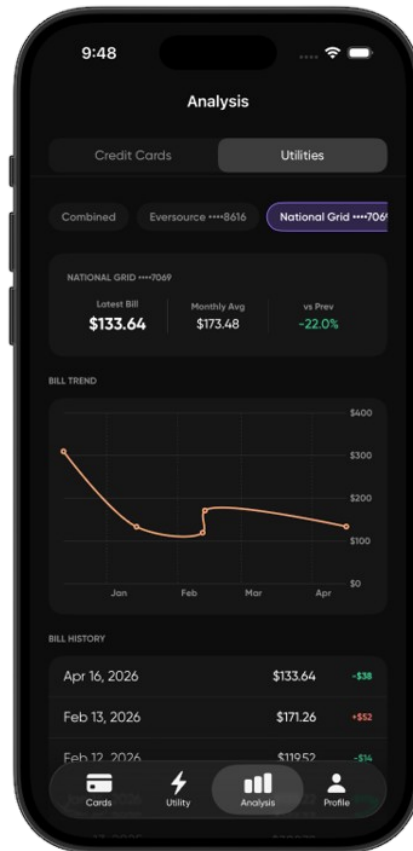


What you can do on this screen:

- Browse every LLM-extracted transaction for the card in date order
- See merchant name, category icon, and posting date per transaction
- Track recurring subscriptions (e.g. OpenAI, Xfinity, Uber) across months
- Cross-reference transactions against the statement period they belong to

Screen 7 — Analytics — Utility Bills

The Analysis tab fetches data from two backend endpoints: GET /analytics/cards and GET /analytics/utilities. Analytics are computed server-side by the AnalyticsWorkerActor, which is sharded by userId across the two-node Akka Cluster — each user's analytics computation is fully isolated from others. Switching to the Utilities segment and selecting a provider (National Grid ●●●●7069) displays the latest bill amount, the rolling monthly average, and the percentage change vs the previous bill — all computed by the actor from the utility_bills table. The line chart plots the bill trend over time. The Bill History list below shows each bill with its month-over-month delta highlighted in green (decrease) or red (increase), extracted from utility statement emails by the UtilityBillCoordinatorActor pipeline.

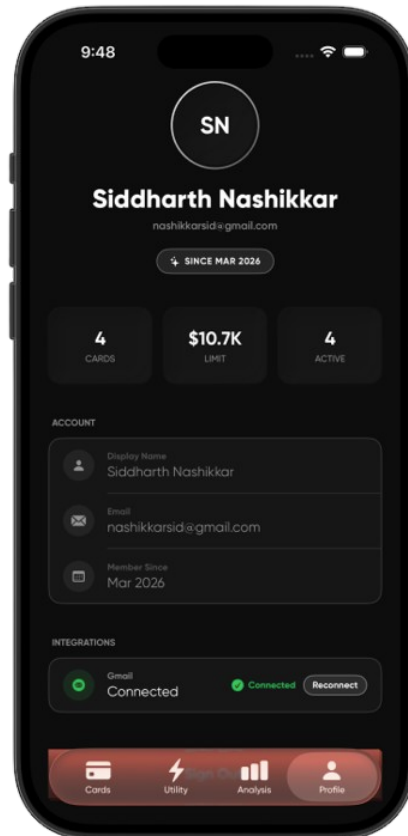


What you can do on this screen:

- Toggle between Credit Card and Utility analytics segments
- Select a specific utility account to see its isolated bill history
- Read the trend chart to spot seasonal usage spikes
- See the exact month-over-month bill delta per entry in Bill History

Screen 8 — Profile & Gmail Integration

The Profile tab shows a summary computed from the user's card data: total cards, combined credit limit, and active card count. Account details (display name, email, member-since date) are stored in the users table and served on login. The Integrations section reflects the Gmail connection state stored in the gmail_credentials table. A Connected badge means the OAuth2 refresh token is valid and encrypted at rest using AES-256-GCM. If the token has expired or been revoked, the badge changes to Disconnected and the Reconnect button calls GET /gmail/oauth/authorize, which builds a new Google consent URL with the gmail.readonly scope and redirects the user through the OAuth2 flow. Once re-authorized, Gmail polling resumes automatically on the next hourly cycle without any further user action.



What you can do on this screen:

- View account summary: total cards, combined credit limit, active count
- See account details: display name, email, and member-since date
- Check Gmail integration status (Connected = valid AES-256-GCM encrypted token)
- Tap Reconnect to re-authorise Gmail OAuth2 and resume automatic statement scanning

6. Core Features Delivered

6.1 Gmail OAuth2 Integration

During onboarding, the app calls GET /gmail/oauth/authorize. GmailOAuthService builds a Google consent URL with scope gmail.readonly and a HMAC-signed state parameter containing the user ID. After the user grants consent, Google redirects to GET /gmail/oauth/callback. The backend verifies the HMAC, exchanges the authorisation code for tokens via `oauth2.googleapis.com/token`, and stores the refresh token in `gmail_credentials` encrypted with AES-256-GCM. The access token is refreshed automatically by the actor system whenever it expires. The iOS Profile tab shows the live connection status from this table.

6.2 LLM Statement & Transaction Extraction

Statement PDFs and email bodies are processed by Llama 3.1 running locally via Ollama. PDFBox extracts raw text from the PDF (capped at 12,000 characters). Spring AI (with Ollama as the provider) sends this text to the model with a prompt that instructs it to return a JSON object containing: statementDate, statementBalance, minimumDue, dueDate, and a transactions array (each with date, merchantName, amount, type). Statements and transactions are extracted in a single LLM call — no second pass needed. The provider is fully abstracted by Spring AI: switching from Ollama to GPT-4o or Claude requires only a single configuration change.

6.3 Event-Sourced Persistence & Crash Recovery

PersistentStatementLedgerActor uses Akka Persistence to write every state change (statement added, transaction added, payment recorded) as an immutable event to the Akka journal. The actor maintains a full in-memory ledger of all statements and transactions. If the process crashes, Akka replays the complete event journal on restart and the actor recovers its exact state — zero data loss. A restartWithBackoff supervisor handles transient failures with exponential back-off from 2 seconds up to 60 seconds.

6.4 Akka Cluster Sharding

The two JVM nodes (the Spring Boot backend and the Akka AI agent) auto-form an Akka Cluster at startup using seed-node configuration — no external orchestrator. AnalyticsWorkerActor is sharded across both nodes by userId using Akka ClusterSharding. This means each user's analytics computation runs on exactly one node, is fully isolated from every other user, and can failover to the other node automatically if one goes down.

6.5 Utility Bill Tracking

Utility bills are tracked through a separate but parallel pipeline: UtilityBillCoordinatorActor polls Gmail for emails from known utility providers (Eversource, National Grid). UtilityBillInitActor handles the historical backfill for newly added utility accounts. UtilityBillExtractorActor uses the same Llama 3.1 pipeline to extract the bill amount, due date, and billing period. Bills can also be added manually via PDF upload — the file is stored in Firebase Storage and PdfExtractionScheduler processes it every 2 minutes. The Analytics tab shows per-provider bill trend charts computed by AnalyticsWorkerActor.

7. Above & Beyond

7.1 Cluster Sharding with Two Live JVM Nodes

- AnalyticsWorkerActor sharded by userId across Node 1 (Spring Boot) and Node 2 (Akka agent)
- Nodes auto-form the cluster using Akka seed-node config — no Kubernetes or ZooKeeper required
- Per-user analytics processing is fully isolated — one user's actor crash cannot affect another
- Cluster membership is monitored; actor location is transparent to the caller

7.2 Event-Sourced Persistence

- PersistentStatementLedgerActor writes every statement and transaction event to the Akka journal
- Full state replay on crash — actor recovers to its exact last state on restart
- restartWithBackoff supervisor: 2 s initial delay, 60 s maximum — no tight failure loops
- In-memory ledger always consistent with the journal — no cache invalidation issues

7.3 historyId Cursor & Zero-Loss Retry

- Gmail History API used incrementally — only messages added since last historyId are fetched
- historyId is stored per-user in gmail_credentials and updated only after successful processing
- If LLM extraction fails for any email, historyId is NOT advanced — the email is automatically retried next cycle
- gmail_message_id unique constraint in card_statements ensures no statement is inserted twice even on retry

7.4 pipeToSelf & Spring AI Provider Abstraction

- All async Ollama HTTP calls are piped back into the actor's own mailbox via pipeToSelf — the actor thread is never blocked
- The actor remains live and can process other messages while waiting for the LLM response
- Spring AI abstracts the LLM provider: Ollama → GPT-4o → Claude requires only a single config-line change
- Init scan (historical backfill) and incremental scan run as separate actor instances to avoid blocking each other

8. Conclusion & Future Work

8.1 What Was Delivered

- Gmail OAuth2 per-user polling with AES-256-GCM encrypted token storage
- Incremental Gmail History API cursor with automatic retry on LLM failure
- Llama 3.1 via Ollama — extracts statements and transactions in a single LLM call
- Event-sourced persistence via PersistentStatementLedgerActor with full crash-replay
- Akka Cluster with AnalyticsWorkerActor sharded across two JVM nodes by userId
- BIN-based card identification (8-digit → 6-digit fallback, no external API)
- SwiftUI iOS app with 25+ screens across Cards, Utility, Analysis, and Profile tabs
- CRED Synth / NeoPop design language for a premium dark-mode financial experience
- Firebase Storage for PDF statements; PdfExtractionScheduler polls every 2 minutes

8.2 Short-Term Enhancements

- Push notifications for upcoming statement due dates
- Budget goals and per-category spend alerts powered by the existing transaction categories
- Statement PDF in-app viewer using the Firebase Storage URL already stored per statement
- Multi-card credit utilization comparison chart on the Analysis tab

8.3 Long-Term Vision

- Plaid / MX direct bank API integration for real-time balance sync without Gmail dependency
- Android app with Jetpack Compose sharing the exact same Spring Boot backend
- Multi-LLM routing — route different bank formats to the best-performing model
- Shared expense splitting with contacts using the extracted transaction data

8.4 Key Takeaway

CredTrack proves that Akka Typed, event sourcing, and cluster sharding are practical choices for a production-grade fintech pipeline — not just academic exercises. The modular design means each layer evolves independently: swap the LLM in one config line, add a new bank email parser in one actor, add a new analytics chart without touching the extraction pipeline. The architecture is built to scale and the iOS client faithfully surfaces all of that complexity behind a clean, CRED-inspired dark-mode interface.